Тема 2

Програмне управління HTML-документом.

Заняття 2/3: Сортування даних за параметром.

1. Визначення параметрів.
2. Сортування даних з масиву.
Основна література: [3], с. 80-85.

1. Визначення параметрів.

JavaScript код зазвичай інтегрується з HTML-документом для того, щоб реалізовувати динамічну поведінку на веб-сторінці. Існує кілька способів це зробити.

Завдання створіть HTML файл та JS документ для відпрацювання КП. Виконайте наступні приклади:

Параметри передаються функції під час її виклику.

```
function greet(name) {
    console.log("Привіт, " + name + "!");
}
```

Визначте параметри за замовчуванням :

```
function greet(name = "Гість") {
    console.log("Привіт, " + name + "!");
}
greet();  // Виведе: Привіт, Гість!
greet("Іван");  // Виведе: Привіт, Іван!
```

Застосуйте оператор залишку (Rest parameters)

Цей оператор дозволяє працювати з невизначеною кількістю параметрів.

```
function sum(...numbers) {
    return numbers.reduce((acc, num) => acc + num, 0);
}
console.log(sum(1, 2, 3, 4));  // Виведе: 10
```

Створіть об'єкт параметрів (Destructuring)

Застосуйте деструктуризацію для отримання окремих властивостей з об'єкта, переданого як параметр.

```
function printUserInfo({ name, age }) {
    console.log(`Ім'я: ${name}, Вік: ${age}`);
}
```

```
const user = { name: "Олена", age: 25, city: "Київ" };
printUserInfo(user);  // Виведе: Ім'я: Олена, Вік: 25
```

Подібно до об'єктів, застосуйте деструкцію в масиві з параметрами.

```
function printFirstTwo([first, second]) {
   console.log(first, second);
}
const numbers = [10, 20, 30];
printFirstTwo(numbers);  // Виведе: 10 20
```

Створіть об'єкт доступний всередині функції та містить усі аргументи, передані функції.

```
function sum() {
   let total = 0;
   for (let i = 0; i < arguments.length; i++) {
      total += arguments[i];
   }
   return total;
}
console.log(sum(1, 2, 3));  // Виведе: 6
```

2. Сортування даних за параметром у JavaScript

Розглянемо сортування масивів об'єктів за певними параметрами. У JavaScript можна сортувати як прості масиви, так і масиви об'єктів, використовуючи функцію `sort()`.

Завдання: За допомогою методу `sort()`, який використовується для сортування елементів масиву на місці, змінюючи його порядок відсортуйте масив.

```
const numbers = [10, 5, 22, 1, 45];
numbers.sort(); // Сортування за замовчуванням (Unicode)
console.log(numbers); // [1, 10, 22, 45, 5] — неправильне сортування для чисел
```

```
// Для чисел необхідно використовувати функцію порівняння
numbers.sort((a, b) => a - b);
console.log(numbers); // [1, 5, 10, 22, 45]
```

Підключіть його до HTML-документу:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript Integration</title>
</head>
<body>
    <h1>Hello World</h1>

    <script src="script.js"></script>
</body>
</html>
```

Сортування масиву об'єктів за параметром

Завдання: виконайте масиву об'єктів за параметром.

Коли ми працюємо з масивами об'єктів, необхідно надавати функцію порівняння, щоб сортувати елементи за певним полем (параметром) об'єкта.

```
const people = [
    { name: 'Alice', age: 25 },
    { name: 'Bob', age: 30 },
    { name: 'Charlie', age: 22 }
];

// Сортування за віком
people.sort((a, b) => a.age - b.age);
console.log(people);
// [{name: 'Charlie', age: 22}, {name: 'Alice', age: 25}, {name: 'Bob', age: 30}]

// Сортування за іменем (в алфавітному порядку)
```

```
people.sort((a, b) => a.name.localeCompare(b.name));

console.log(people);

// [{name: 'Alice', age: 25}, {name: 'Bob', age: 30}, {name: 'Charlie', age: 22}]
```

Сортування за кількома параметрами

Можна також сортувати масиви за кількома параметрами, наприклад, спочатку за віком, а потім за іменем, якщо вік однаковий.

Завдання: виконайте сортування за кількома параметрами

```
const people = [
   { name: 'Alice', age: 25 },
   { name: 'Bob', age: 30 },
   { name: 'Charlie', age: 22 },
   { name: 'Charlie', age: 30 }
];

// Спочатку сортуємо за віком, потім за іменем
people.sort((a, b) => {
   if (a.age === b.age) {
      return a.name.localeCompare(b.name);
   }
   return a.age - b.age;
});
console.log(people);
// [{name: 'Charlie', age: 22}, {name: 'Alice', age: 25}, {name: 'Bob', age: 30}, {name: 'Charlie', age: 30}]
```

Реверсне сортування

Для сортування у зворотному порядку можна змінити знак порівняння або використовувати метод `reverse()` після сортування.

Завдання: виконайте зворотнє сортування

```
const people = [
   { name: 'Alice', age: 25 },
   { name: 'Bob', age: 30 },
   { name: 'Charlie', age: 22 }
];

// Сортування у зворотному порядку за віком
```

```
people.sort((a, b) => b.age - a.age);
console.log(people);
// [{name: 'Bob', age: 30}, {name: 'Alice', age: 25}, {name: 'Charlie', age: 22}]
```

Сортування складних даних

Якщо у вас є складніші структури даних, сортування можна виконувати за кількома рівнями вкладеності.

Завдання: виконайте сортування

```
const students = [
    { name: 'Alice', scores: { math: 85, english: 90 } },
    { name: 'Bob', scores: { math: 95, english: 85 } },
    { name: 'Charlie', scores: { math: 85, english: 88 } }
];

// Сортування за оцінками з математики
students.sort((a, b) => a.scores.math - b.scores.math);
console.log(students);
// [{name: 'Alice', scores: {math: 85, english: 90}}, {name: 'Charlie', scores: {math: 85, english: 88}}, {name: 'Bob', scores: {math: 95, english: 85}}]
```

Завдання для додаткового виконання

- **Завдання 1: Сортування числового масиву**

Створіть масив чисел. Використовуючи метод `sort()`, відсортуйте його спочатку за зростанням, а потім за спаданням.

- **Завдання 2: Сортування масиву об'єктів за параметром**

Створіть масив об'єктів, кожен з яких містить інформацію про людину (ім'я, вік). Відсортуйте масив за віком спочатку за зростанням, потім — в алфавітному порядку за іменем.

- **Завдання 3: Сортування за кількома параметрами**

Створіть масив об'єктів, що містить інформацію про студентів (ім'я, вік, середній бал). Відсортуйте їх за середнім балом. Якщо середній бал однаковий — сортуйте за віком.

- **Завдання 4: Сортування вкладених даних**

Створіть масив об'єктів, кожен з яких містить інформацію про студентів та їх оцінки з кількох предметів. Відсортуйте їх за оцінками з одного з предметів (наприклад, математика).

Хід виконання роботи

1. Створити HTML-документ index.html та окремий JavaScript-файл script.js. Підключити JavaScript-файл до HTML-документа.
2. Ознайомитися зі способами передавання параметрів у функції JavaScript. Виконати наведені приклади використання параметрів за замовчуванням, оператора Rest, деструктуризації об’єктів і масивів та об’єкта arguments.
3. Дослідити роботу методу sort() для сортування числових масивів. Порівняти результати сортування за замовчуванням і з використанням функції порівняння.
4. Створити масив об’єктів та реалізувати його сортування за числовими й текстовими властивостями.
5. Виконати сортування масиву об’єктів за кількома параметрами, передбачивши додаткову умову порівняння для однакових значень.
6. Реалізувати сортування масиву у зворотному порядку.
7. Створити масив об’єктів із вкладеними даними та виконати сортування за однією з вкладених властивостей.
8. Виконати додаткові завдання із сортування числових масивів, масивів об’єктів та вкладених структур даних.
9. Перевірити правильність роботи JavaScript-коду за допомогою консолі розробника браузера. Проаналізувати отримані результати.

Вимоги до звіту

Звіт повинен містити:

1. Тему та мету практичної роботи.
2. Короткі теоретичні відомості про параметри функцій та метод sort() у JavaScript.
3. Лістинг HTML-документа та JavaScript-коду, реалізованого під час виконання завдань.
4. Скріншоти результатів виконання прикладів у консолі браузера.
5. Результати сортування масивів за різними параметрами із зазначенням початкового та відсортованого порядку елементів.
6. Лістинг програмного коду та результати виконання додаткових завдань.
7. Висновки за результатами виконаної роботи.

Контрольні запитання

1. Які способи підключення JavaScript до HTML-документа існують?
2. Що таке параметри та аргументи функції в JavaScript? У чому їх відмінність?
3. Для чого використовуються параметри функції за замовчуванням?

4. Яке призначення оператора Rest (...) у параметрах функції?
5. Що таке деструктуризація об’єктів і масивів у JavaScript?
6. Для чого використовується об’єкт arguments?
7. Яке призначення методу sort() у JavaScript?
8. Чому сортування числового масиву методом sort() без функції порівняння може давати неправильний числовий порядок?
9. Як реалізувати сортування числового масиву за зростанням і спаданням?
10. Як виконати сортування масиву об’єктів за визначеною властивістю?
11. Для чого використовується метод localeCompare()?
12. Як реалізувати сортування масиву об’єктів за кількома параметрами?
13. У чому відмінність між методами sort() та reverse()?
14. Як виконати сортування масиву об’єктів за вкладеною властивістю?
15. Чи змінює метод sort() початковий масив? Як можна зберегти його початковий порядок?